Vision-based Dirt Detection for Cleaning Robot

Shufang Tao (3824652), Yuxin Song (3820779), and Yuyang Ren (3762594)

***Abstract*— Visual dirt detection allows a cleaning robot to distinguish removable dirt from objects that should be avoided or reported. This project reproduces the detection tasks of DirtNet without robot control or ROS and evaluates modern YOLO models for two operating modes: binary detection of *dirt* versus *object*, and ten-class detection of dirt and nine office-item categories. Several YOLO families were screened before YOLO11 was selected for detailed optimization. On the full binary dataset, increasing the input size to 1280 pixels and training for 30 epochs produced validation mAP@0.5 values of 0.979 for YOLO11s and 0.983 for YOLO11m. However, their performance on 80 held-out real images decreased to 0.806 and 0.778, respectively. A ten-class YOLO11m baseline achieved 0.961 mAP@0.5 on synthetic validation data and 0.737 on the real test set. Additional low-rate fine-tuning improved synthetic validation performance but did not improve real-image transfer. The experiments establish functional binary and multiclass detection pipelines, while showing that synthetic-to-real domain mismatch, rather than insufficient model capacity, is the main remaining limitation.**

I. Introduction

Professional cleaning robots operate in environments where dirt, litter, and office items may appear unexpectedly. A useful visual system must detect small and irregular dirt regions, but it should also avoid treating valuable objects such as keys, pens, or notebooks as removable dirt. Reliable visual detection can support demand-oriented cleaning, cleaning-result verification, and the localization of objects left on the floor.

The original DirtNet system addressed this problem with an adapted YOLOv3 detector and a generator for synthetic floor scenes [1]. The present project reproduces its detection tasks in a simplified setting. Robot navigation, cleaning control, and ROS integration are outside the scope. Instead, the work focuses on three experimental questions:

1) Which YOLO family and model size provide a suitable accuracy–cost trade-off for floor-object detection?
2) Which training settings meaningfully improve YOLO11 binary detection?
3) Do improvements on synthetic validation images transfer to real images and to the more difficult ten-class task?

The main contributions are a unified binary architecture screening, a systematic YOLO11 optimization on the full binary dataset, a ten-class extension, and an evaluation of the synthetic-to-real generalization gap. Final models are also demonstrated on phone video recorded from a robot-like viewpoint.

II. Background

A. DirtNet and YOLO

DirtNet combines synthetic image generation with a modified YOLOv3 detector [1], [2]. Its generator blends dirt and office item samples with clean floor images, reducing the effort required to collect and annotate complete scenes. The paper defines both a two-class dirt/object task and a ten-class task. It also reports that a synthetic dataset size of 9,248 images gives a favorable balance between training cost and detection performance.

Modern YOLO detectors follow the same high-level pipeline. A backbone extracts hierarchical visual features, a neck fuses features at multiple scales, and the detection head predicts bounding boxes, class labels, and confidence scores. This multi-scale design is important here because dirt spots may occupy only a few pixels, while notebooks and rulers cover much larger image regions. Our implementation uses pretrained models and the Ultralytics training interface [3].

B. Evaluation Measures

Precision measures the fraction of predicted detections that are correct, while recall measures the fraction of ground-truth instances that are found. Average precision (AP) summarizes the precision–recall curve for one class. Mean average precision at an intersection-over-union threshold of 0.5 is denoted mAP@0.5. mAP@0.5:0.95 averages over thresholds from 0.5 to 0.95 and therefore evaluates box localization more strictly. Binary architecture screening primarily uses mAP@0.5, whereas the multiclass comparisons also emphasize mAP@0.5:0.95. Inference time is reported separately because real-time operation is relevant to a moving robot.

III. Concept and Implementation

A. Tasks and Datasets

The binary task maps every original label except *dirt* to a common *object* class. The ten-class task retains the labels *dirt*, *business cards*, *keys*, *other*, *paper and notebooks*, *pens*, *rulers*, *scissors*, *tapes*, and *USB sticks*. Table I summarizes the datasets. The full binary train/validation set contains eight times as many images as the initial small set. The 80 real images and their 644 boxes remain unchanged and are never included in training.

Training and validation data are synthetic or blended floor scenes, whereas `test_real_new` contains real photographs. The real set has no background-only image. Consequently, it can measure missed and incorrectly classified labeled objects, but it cannot fully estimate false-alarm rates during continuous operation on clean floors.

TABLE I

DATASET SPLITS USED IN THE PROJECT.

Dataset	Train img.	Val. img.	Train boxes	Val. boxes
Binary small / multiclass	860	296	8,626	2,962
Binary full	6,880	2,368	68,955	23,597
Real test (held out)	80 images		644 boxes	

Figure 1 illustrates both the task and the domain difference. The blended training image has sharply composited objects and controlled floor textures, while the independent real image contains darker illumination, reflections, and less regular object boundaries. The staged scene further shows why a cleaning system must separate removable dirt-like fragments from objects that should remain on the floor. These visual differences motivated reporting synthetic validation and real-test results separately throughout the project.

B. Dataset Conversion

The source annotations store axis-aligned boxes as absolute corner coordinates. The conversion script clips each box to its image boundary, rejects degenerate boxes, and writes normalized YOLO center coordinates and box dimensions. For the binary task, *dirt* is assigned class 0 and every other source class is assigned class 1. Images without an annotated object retain an empty label file, so background examples are not silently discarded.

The train/validation partition is determined from the floor identifier encoded in each filename: floors 0–15 form the training split and floors 16–20 form the validation split. This keeps images derived from one floor identifier in a single split. The converter also preserves the predefined real and synthetic test folders and generates YOLO dataset descriptions with relative paths. The resulting directory pairs every `images/<split>` folder with a corresponding `labels/<split>` folder, allowing the same data layout to be used by every Ultralytics model tested in the project.

C. Experimental Pipeline

The work proceeds in four stages. First, multiple YOLO versions are screened on the simpler binary task with both native and aligned settings. Second, the selected YOLO11 family is evaluated at different scales and optimized on the full binary dataset. Third, the most relevant architectures are validated on the ten-class task, followed by a controlled YOLO11m baseline and fine-tuning experiment. Finally, selected models are evaluated on real images and on robot-like video.

The distinction between model selection and final testing is maintained in the main experiments. Validation metrics select checkpoints and formal candidates. The real set is then used to expose deployment behavior. Later checkpoint sweeps on this same real set are explicitly treated as post-hoc diagnostics rather than as independent model-selection evidence.

Training is launched through the Ultralytics command-line interface using a model checkpoint and a dataset YAML file. Validation is executed at the end of every epoch, and both `best.pt` and `last.pt` are retained. Formal evaluations reload the selected checkpoint at the training image size and export precision, recall, mAP, timing, curves, and confusion matrices. Prediction mode is used only for qualitative images and the final phone-video demonstration. This separation keeps training, quantitative evaluation, and presentation artifacts reproducible without modifying the Ultralytics package source.

TABLE II

ALIGNED BINARY ARCHITECTURE SCREENING ON THE SMALL DATASET.

Model	Precision	Recall	mAP50	mAP50–95	Time (min)
YOLOv3u	0.933	0.917	**0.952**	0.790	71.4
YOLO11l	**0.938**	0.903	0.950	**0.798**	**40.7**
YOLOv8l	0.936	0.894	0.949	0.771	46.4
YOLOv5lu	0.937	0.900	0.945	0.777	42.0
YOLO26l	0.936	0.897	0.944	0.764	46.3

D. Training Platforms

The binary experiments were performed on a desktop NVIDIA GeForce RTX 4070. The formal ten-class YOLO11m experiments ran on an NVIDIA GeForce RTX 3060 Laptop GPU with 6 GB memory. The multiclass log records Python 3.12.10, PyTorch 2.12.1+cu130, and Ultralytics 8.4.74. Model weights were initialized from pretrained checkpoints. A fixed seed of zero and deterministic mode were used in the formal YOLO11 runs.

Because the binary and multiclass experiments used different GPUs, absolute training times are not compared across these two groups. Time comparisons are only used among runs performed within the same experimental setup. GPU memory also constrained the final YOLO11m binary run: at 1280 pixels, batch size 8 was too expensive, so batch size 4 was used. The effective parameters of each run are stored in `args.yaml`; metrics and curves are stored in `results.csv` and image artifacts.

IV. EXPERIMENTS AND EVALUATION

A. Architecture Screening

The initial binary comparison showed that configuration differences can alter a ranking. With native settings, the original YOLOv3 reproduction remained strong at 0.946 mAP@0.5. Table II shows the large-model comparison after aligning training length and the main optimizer settings. YOLOv3u achieved the highest mAP@0.5, but YOLO11l was within 0.002 while requiring substantially less training time. YOLO11 was therefore selected as the main family for further binary optimization.

Model size was first studied within the YOLOv8 family on the ten-class task. YOLOv8n, YOLOv8s, and YOLOv8m obtained best validation mAP@0.5:0.95 values of 0.710, 0.796, and 0.813, respectively. The large gain from nano to small and the smaller gain from small to medium indicated that nano capacity was insufficient, while returns diminished at larger scales. A subsequent aligned comparison of YOLOv5lu, YOLOv8l, and YOLO11l produced



Fig. 1. Representative project images. Left: blended training image. Center: independent real-test image. Right: staged mixed-object floor scene. The change in lighting, texture, reflections, and boundary appearance illustrates the synthetic-to-real domain shift.

TABLE III

SELECTED YOLO11 BINARY OPTIMIZATION RESULTS ON VALIDATION DATA.

Model	Setting	P	R	mAP50	mAP50–95	min
11s	default, 1024, 15e	0.950	0.917	0.956	0.834	34.3
11s	SGD, 1024, 15e	0.963	0.923	0.959	0.844	34.9
11s	SGD, 1024, 30e	0.961	0.934	0.962	0.872	67.3
11m	SGD, 1024, 15e	0.965	0.939	0.971	0.875	68.6
11s	SGD, 1280, 15e	**0.976**	**0.944**	**0.977**	**0.876**	53.5
11s	+ rect, 1280, 15e	0.969	0.941	0.975	0.868	42.6

mAP@0.5:0.95 values of 0.820, 0.824, and 0.832. YOLO11l also had the shortest training time in that comparison. YOLOv3u was planned but could not be completed under the available GPU memory limit. These results confirmed YOLO11 as the most suitable family among the completed multiclass candidates. For the controlled baseline on the 6 GB laptop GPU, YOLO11m was then chosen as a practical capacity–memory compromise.

B. YOLO11 Binary Optimization

YOLO11n, YOLO11s, and YOLO11m were first trained for 15 epochs at 1024 pixels on the full binary dataset. Their mAP@0.5 values were 0.929, 0.956, and 0.962, while training took 21.5, 34.3, and 69.3 minutes. YOLO11n was clearly weaker. YOLO11m was more accurate, but required about twice the time of YOLO11s. Consequently, YOLO11s was used for parameter screening and stable choices were later transferred to YOLO11m.

Table III contains the most informative binary experiments. SGD improved all main metrics relative to the short default run. Extending training to 30 epochs mainly improved recall and strict localization quality. Increasing the input size from 1024 to 1280 was the strongest single change: the 15-epoch high-resolution run exceeded the 30-epoch 1024 run. This supports the hypothesis that preserving detail is important for small dirt regions.

Removing additional augmentation options did not change the 1024-pixel SGD result, and disabling automatic mixed precision increased training time by more than 50% for almost no mAP@0.5 gain. Both directions were rejected. Rectangular training at 1280 reduced time by about 20%, but slightly reduced the metrics. In the current Ultralytics pipeline, `rect=True` also disables mosaic. A separate no-mosaic run showed a similar mAP@0.5 decrease, suggesting that much of the loss came from removing mosaic rather than from rectangular batching itself. This is related to, but not identical to, the fixed rectangular 1024×832 input used in the original DirtNet implementation. A cosine learning-rate schedule did not improve the 15-epoch 1280-pixel baseline, so the default schedule was retained.

C. Final Binary Models and Real Transfer

The two final candidates used SGD, default detection augmentation, mixed precision, 1280-pixel input, and 30 epochs. YOLO11s used batch size 8; YOLO11m used batch size 4 because of memory pressure. Table IV shows both synthetic validation and held-out real-test results.

Validation selected YOLO11m: it had higher recall, mAP@0.5, and particularly mAP@0.5:0.95. The real test reversed most of this ranking. YOLO11m remained more precise, but YOLO11s had better recall, both mAP values, and approximately twice the inference speed. The mAP@0.5 decrease from validation to real data was 0.173 for YOLO11s and 0.205 for YOLO11m. The corresponding strict-mAP decreases were 0.344 and 0.391.

A post-hoc sweep gave additional evidence about this discrepancy. A simpler YOLO11s run at 1024 pixels and 15 epochs reached 0.885 real-test mAP@0.5, while a one-epoch run already reached 0.875. Testing every saved checkpoint produced a non-monotonic real-test curve and a maximum of 0.895 near the end of the 15-epoch run. Since these results repeatedly reuse `test_real_new`, they are not used to replace the formally validation-selected model. They serve only as a diagnostic indication that short fine-tuning preserves generic pretrained features, whereas stronger adaptation may specialize to synthetic textures.

D. Ten-Class Detection

The formal multiclass baseline used YOLO11m with 1024-pixel input, batch size 2, automatic optimizer selection, mixed precision, and 30 epochs. The best checkpoint occurred at epoch 29. Figure 2 shows stable convergence on

TABLE IV

FINAL BINARY VALIDATION AND REAL-TEST RESULTS. INFERENCE IS MEASURED ON THE REAL TEST SET.

		Synthetic validation				Real test				
Model	Setting	P	R	mAP50	mAP50–95	P	R	mAP50	mAP50–95	ms/img
YOLO11s	1280, 30e, batch 8	0.972	0.951	0.979	0.906	0.781	**0.772**	**0.806**	**0.562**	**10.0**
YOLO11m	1280, 30e, batch 4	**0.974**	**0.962**	**0.983**	**0.923**	**0.835**	0.704	0.778	0.532	21.4

TABLE V

MULTICLASS SYNTHETIC-VALIDATION AND REAL-TEST COMPARISON.

Experiment	Domain	P	R	mAP50	mAP50–95
Baseline	synthetic val.	0.953	0.930	0.961	0.814
Baseline	real test	0.665	**0.776**	**0.737**	0.554
Fine-tuned	synthetic val.	**0.967**	**0.944**	**0.968**	**0.826**
Fine-tuned	real test	**0.726**	0.721	0.726	**0.557**

the synthetic data and the per-class precision–recall behavior on real images.

Table V summarizes the baseline and a second-stage fine-tuning run. The baseline achieved 0.961 mAP@0.5 and 0.814 mAP@0.5:0.95 on synthetic validation, but only 0.737 and 0.554 on real images. Pens, scissors, paper/notebooks, and business cards were the strongest real classes. The weakest were *other*, rulers, and USB sticks. For dirt specifically, real-test precision was 0.852, recall was 0.706, mAP@0.5 was 0.773, and mAP@0.5:0.95 was 0.489. Irregular dirt boundaries therefore remained difficult at strict IoU thresholds.

Fine-tuning started from the baseline best checkpoint and added 15 epochs with AdamW, an initial learning rate of 10^{-4}, cosine decay, patience 5, and mosaic disabled. Every synthetic validation metric improved. On real images, however, precision increased while recall fell by 0.055 and mAP@0.5 decreased by 0.011. The nearly unchanged mAP@0.5:0.95 does not represent a meaningful transfer gain. This experiment directly shows that additional optimization on the same synthetic distribution cannot solve the remaining real-domain problem.

E. Comparison with DirtNet and Qualitative Demo

On the original real test, DirtNet-D and DirtNet-M reported mAP@0.5 values of 0.855 and 0.838 for the two-class task, and 0.807 and 0.771 for the ten-class task [1]. The formal final YOLO11s and YOLO11m binary candidates reached 0.806 and 0.778, while the YOLO11m multiclass baseline reached 0.737. These numbers show that the modern training pipeline was successfully reproduced, but it did not consistently surpass the domain-specific original DirtNet on real images. The comparison should not be interpreted as a strict architecture benchmark because the implementations, model-selection procedures, and hardware differ.

Figure 3 compares binary and multiclass outputs on the same phone-video frame. The binary model covers the visible items with a coarse dirt/object decision. The multiclass model provides more useful semantic labels, but also illustrates confusions among visually similar categories. For example, some ruler- or key-like items are assigned to paper/notebooks or the broad *other* class.

Several motion patterns were tested while recording the demo. Straight and stable movement gave acceptable detections, while large rotations introduced motion blur and unstable boxes. Slow movement with short pauses kept objects visible for more frames and improved detection stability. The final video was selected by balancing detection behavior, object coverage, visual clarity, and presentation quality.

V. DISCUSSION

The binary and multiclass experiments lead to the same central finding: synthetic validation performance is not a reliable proxy for real-image performance in the current data setup. Training and validation images share the same generation process, object samples, and labeling style. Real images differ in illumination, sensor noise, reflections, floor texture, object material, dirt shape, and boundary clarity. A model can therefore improve on the validation distribution while relying more strongly on details that do not transfer.

Higher resolution improved synthetic validation because it preserved small dirt features, but it also gave the detector more access to texture differences between synthetic and real floors. Larger models can similarly fit domain-specific patterns more strongly. This interpretation is consistent with the strong one-epoch and 15-epoch real-test results and with the failed multiclass fine-tuning attempt. It does not prove that short training is universally optimal; it shows that training duration cannot be selected reliably without a representative real validation set.

Several limitations constrain the conclusions. The real test contains only 80 images, so class-level estimates have high variance. It contains no clean, background-only scenes and thus does not fully quantify deployment false alarms. The *object* binary class and the *other* multiclass category cover broad visual distributions. Some classes, particularly rulers, tapes, and scissors, are underrepresented in training. Finally, repeated post-hoc inspection of the real test weakens its status as a strictly untouched test set. Future work should create separate real training, validation, and final-test splits rather than continue to tune against these 80 images.

VI. CONCLUSION AND OUTLOOK

This project established reproducible binary and ten-class YOLO detection pipelines for floor dirt and office items. Architecture screening selected YOLO11 as a strong accuracy–cost compromise. For binary detection, SGD, high-resolution input, mixed precision, default augmentation, and longer final training produced the best synthetic validation models.

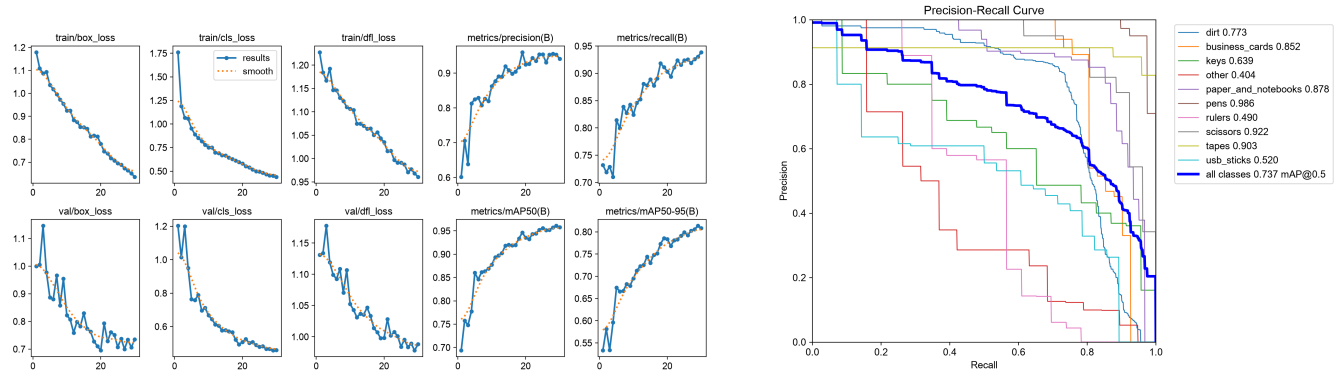


Fig. 2. Ten-class YOLO11m baseline. Left: training and synthetic-validation curves. Right: per-class precision–recall curves on the independent real test set. Synthetic convergence is strong, while real performance varies substantially between classes.

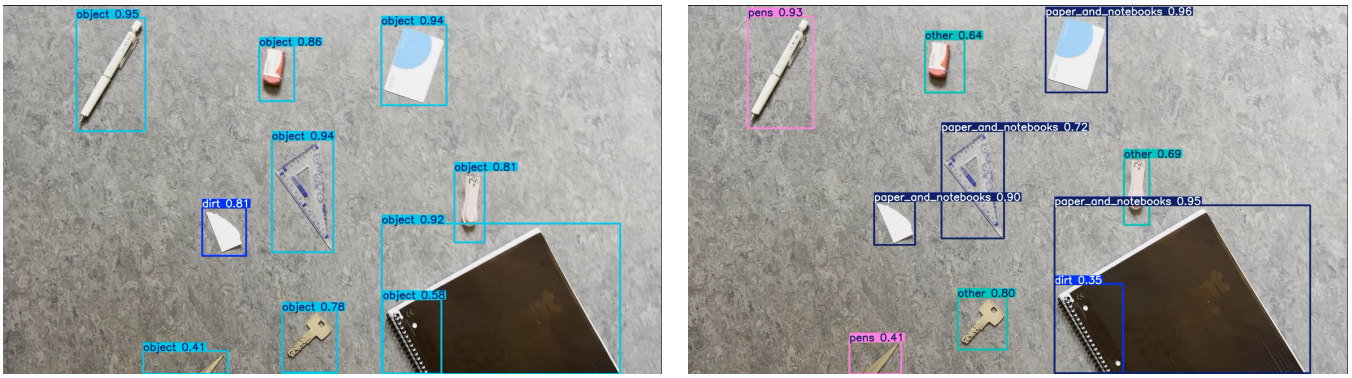


Fig. 3. Qualitative detections on the same robot-like video frame. Left: binary dirt/object detection. Right: ten-class detection. Binary detection is coarser, whereas multiclass detection adds semantic information but exposes category ambiguities.

YOLO11m was validation-selected, while YOLO11s gave a better real-test balance and faster inference. The ten-class YOLO11m baseline also converged strongly on synthetic validation data, but its real performance remained below both its validation score and the original DirtNet results.

The next improvement should therefore target data rather than additional synthetic-only optimization. Separate real training and validation images, clean floor negatives, reflections, shadows, scratches, low-light examples, and more samples of weak classes are needed. The definition of *other* should be reconsidered, and confidence thresholds should reflect the different costs of missing dirt and falsely identifying valuable objects. After real-domain adaptation, the detector can be integrated with robot motion and cleaning decisions.

References

[1] R. Bormann, X. Wang, J. Xu, and J. Schmidt, "DirtNet: Visual Dirt Detection for Autonomous Cleaning Robots," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2020.

[2] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[3] Ultralytics, "Ultralytics YOLO," GitHub repository. [Online]. Available: `https://github.com/ultralytics/ultralytics`